18.5 回复塑形与抑制

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~520k tokens，输出 ~45k tokens（本章合计）

模型产生的原始文本并不能直接发送给用户。在"流式传输 → 分块 → 发送"这条管线的最后阶段，OpenClaw 还需要完成一系列**回复塑形（Reply Shaping）**操作：过滤静默标记、去除重复消息、将工具执行摘要内联到对话中。这些操作共同保证了用户收到的消息既干净又完整。

18.5.1 `NO_REPLY` 静默令牌过滤

问题背景

在某些场景下，AI Agent"听到"了消息但不需要回复——例如群聊中他人之间的对话、心跳检测、或者用户设置了自动回复条件未满足的情况。此时模型仍然会被调用（以维持上下文连贯），但它产出的文本应该被**完全丢弃**，不发送给用户。

OpenClaw 用一个特殊的**静默令牌（Silent Reply Token）**来实现这一机制。

令牌定义

静默令牌定义在 `src/auto-reply/tokens.ts` 中，极其简洁：

```
// src/auto-reply/tokens.ts
export const HEARTBEAT_TOKEN = "HEARTBEAT_OK";
export const SILENT_REPLY_TOKEN = "NO_REPLY";
```

系统提示词会指导模型：当不需要回复时，只输出 `NO_REPLY` 即可。与之类似的 `HEARTBEAT_OK` 用于心跳场景——定时检查 AI 是否仍在正常运行。

检测逻辑

`isSilentReplyText` 函数负责判断一段文本是否包含静默令牌：

```ts
// src/auto-reply/tokens.ts
export function isSilentReplyText(
  text: string | undefined,
  token: string = SILENT_REPLY_TOKEN,
): boolean {
  if (!text) return false;

  const escaped = escapeRegExp(token);

  // 检查前缀位置：文本以 NO_REPLY 开头（允许前导空白）
  const prefix = new RegExp(`^\\s*${escaped}(?=$|\\W)`);
  if (prefix.test(text)) return true;

  // 检查后缀位置：文本以 NO_REPLY 结尾
  const suffix = new RegExp(`\\b${escaped}\\b\\W*$`);
  return suffix.test(text);
}
```

这个检测函数有两个地方要注意：

1. **双位置检测**：不仅检查文本开头，还检查结尾。这是因为某些模型可能在 `NO_REPLY` 之前或之后添加空格、标点或换行。
2. **单词边界感知**：使用 `\b` （word boundary）和 `\W` （非单词字符）确保不会误匹配——例如 `NO_REPLY_NEEDED` 不会被当作静默令牌。

> **衍生解释：正则表达式的单词边界 `\b`**
>
> 在正则表达式中， `\b` 匹配"单词字符"与"非单词字符"之间的位置（不消耗字符）。单词字符包括字母、数字和下划线 `[a-zA-Z0-9_]` 。例如在字符串 `"NO_REPLY done"` 中， `\bNO_REPLY\b` 可以匹配 `NO_REPLY` ，因为它的左边是字符串开头（等效于边界），右边是空格（非单词字符）。但 `\bNO_REPLY\b` 不会匹配 `"NO_REPLY_NEEDED"` 中的 `NO_REPLY` ，因为 `Y` 右边的 `_` 是单词字符，不构成边界。

在回复指令解析中的应用

静默令牌的检测被集成到 `parseReplyDirectives` 函数中，这是回复处理的核心入口：

```ts
// src/auto-reply/reply/reply-directives.ts（简化）
export function parseReplyDirectives(
  raw: string,
  options: { currentMessageId?: string; silentToken?: string } = {},
): ReplyDirectiveParseResult {
  const split = splitMediaFromOutput(raw);       // 分离媒体 URL
  let text = split.text ?? "";

  const replyParsed = parseInlineDirectives(text, {
    currentMessageId: options.currentMessageId,
    stripReplyTags: true,                          // 处理 [[reply:xxx]] 指令
  });
  if (replyParsed.hasReplyTag) {
    text = replyParsed.text;
  }

  // 静默检测
  const silentToken = options.silentToken ?? SILENT_REPLY_TOKEN;
  const isSilent = isSilentReplyText(text, silentToken);
  if (isSilent) {
    text = "";                                     // 清空文本，阻止发送
  }

  return {
    text,
    mediaUrls: split.mediaUrls,
    replyToId: replyParsed.replyToId,
    replyToCurrent: replyParsed.replyToCurrent,
    replyToTag: replyParsed.hasReplyTag,
    isSilent,
  };
}
```

`parseReplyDirectives` 的返回值中，`isSilent` 标志和被清空的 `text` 共同作用——上层代码在看到空文本和无媒体 URL 时，会自然跳过该消息的发送。

整个静默令牌的处理流程可以概括为：

```
模型输出: "NO_REPLY"
    │
    ▼
parseReplyDirectives()
    ├── splitMediaFromOutput() → text="NO_REPLY", media=[]
    ├── parseInlineDirectives() → 无特殊指令
    └── isSilentReplyText("NO_REPLY") → true
         │
         ▼
    text = ""           // 清空
    isSilent = true     // 标记
    │
    ▼
上层: if (!text && !media) → 不发送
```

18.5.2 消息工具去重

问题背景

OpenClaw 的 AI Agent 可以通过**消息工具（Messaging Tool）主动向用户或其他渠道发送消息——例如使用 `message` 工具向 Discord 频道发送一条通知。问题在于：模型在使用 `message(action="send", to="user", content="Hello!")` 工具后，往往还会在其最终文本回复中重复同样的内容（"Hello!"）。如果不做去重，用户会收到两条相同的消息**：一条来自工具调用，一条来自最终回复。

消息工具识别

首先需要知道哪些工具属于"消息工具"。这由 `isMessagingTool` 函数判定：

```
// src/agents/pi-embedded-messaging.ts
const CORE_MESSAGING_TOOLS = new Set(["sessions_send", "message"]);

export function isMessagingTool(toolName: string): boolean {
  if (CORE_MESSAGING_TOOLS.has(toolName)) {
    return true;
  }
  // 通道插件也可以注册为消息工具
  const providerId = normalizeChannelId(toolName);
  return Boolean(providerId && getChannelPlugin(providerId)?.actions);
}
```

核心消息工具有两个：

- `sessions_send` ： 会话内发送消息
- `message` ： 通用消息发送（支持指定 provider、channel、to）

此外，任何注册了 `actions` 的通道插件（如 Discord、Slack）也被视为消息工具——它们的工具名本身就是通道 ID（如 `discord` 、 `slack` ）。

发送动作判定

不是所有消息工具调用都是"发送"—— `message` 工具还支持 `read` 、 `list` 等动作。只有发送动作才需要去重：

```
// src/agents/pi-embedded-messaging.ts（简化）
export function isMessagingToolSendAction(
  toolName: string,
  args: Record<string, unknown>,
): boolean {
  const action = typeof args.action === "string" ? args.action.trim() : "";

  if (toolName === "sessions_send") return true;  // 永远是发送
  if (toolName === "message") {
    return action === "send" || action === "thread-reply";
  }

  // 通道插件委托给插件自己判定
  const plugin = getChannelPlugin(normalizeChannelId(toolName));
  return Boolean(plugin?.actions?.extractToolSend?.({ args })?.to);
}
```

追踪与提交机制

消息工具去重用了一个**“先暂存、后提交”**的两阶段设计。这样做的原因很直接：如果工具调用失败了，消息实际上并没有发出去，此时就不应该抑制最终回复中的相同文本。

在 `handleToolExecutionStart` 中，文本被暂存到 `pendingMessagingTexts`：

```
// src/agents/pi-embedded-subscribe.handlers.tools.ts（简化）
export async function handleToolExecutionStart(ctx, evt) {
  // ...
  if (isMessagingTool(toolName)) {
    const isMessagingSend = isMessagingToolSendAction(toolName, argsRecord);
    if (isMessagingSend) {
      // 提取发送目标
      const sendTarget = extractMessagingToolSend(toolName, argsRecord);
      if (sendTarget) {
        ctx.state.pendingMessagingTargets.set(toolCallId, sendTarget);
      }
      // 暂存消息文本
      const text = argsRecord.content ?? argsRecord.message;
      if (text && typeof text === "string") {
        ctx.state.pendingMessagingTexts.set(toolCallId, text);
      }
    }
  }
}
```

在 `handleToolExecutionEnd` 中，根据执行结果决定是提交还是丢弃：

```
// src/agents/pi-embedded-subscribe.handlers.tools.ts（简化）
export function handleToolExecutionEnd(ctx, evt) {
  const isToolError = Boolean(evt.isError) || isToolResultError(result);

  // 从暂存区取出
  const pendingText = ctx.state.pendingMessagingTexts.get(toolCallId);
  if (pendingText) {
    ctx.state.pendingMessagingTexts.delete(toolCallId);
    if (!isToolError) {
      // 成功：提交到已发送列表
      ctx.state.messagingToolSentTexts.push(pendingText);
      ctx.state.messagingToolSentTextsNormalized.push(
        normalizeTextForComparison(pendingText)
      );
      ctx.trimMessagingToolSent();
    }
    // 失败：直接丢弃，不抑制后续回复
  }
}
```

文本归一化与模糊匹配

去重比较不是简单的字符串相等——模型可能在回复中稍微修改措辞（加 emoji、改大小写等）。OpenClaw 使用**归一化比较**：

```ts
// src/agents/pi-embedded-helpers/messaging-dedupe.ts
const MIN_DUPLICATE_TEXT_LENGTH = 10;

export function normalizeTextForComparison(text: string): string {
  return text
    .trim()
    .toLowerCase()
    .replace(/\p{Emoji_Presentation}|\p{Extended_Pictographic}/gu, "")  // 去 emoji
    .replace(/\s+/g, " ")                                               // 合并空白
    .trim();
}

export function isMessagingToolDuplicateNormalized(
  normalized: string,
  normalizedSentTexts: string[],
): boolean {
  if (normalizedSentTexts.length === 0) return false;
  if (!normalized || normalized.length < MIN_DUPLICATE_TEXT_LENGTH) return false;

  return normalizedSentTexts.some((sent) => {
    if (!sent || sent.length < MIN_DUPLICATE_TEXT_LENGTH) return false;
    // 子串包含检测（双向）
    return normalized.includes(sent) || sent.includes(normalized);
  });
}
```

归一化步骤：

步骤	输入示例	输出示例
原始文本	`"  Hello 👋  WORLD 🌍  "`	—
`trim()`	`"Hello 👋  WORLD 🌍"`	—
`toLowerCase()`	`"hello 👋  world 🌍"`	—
去 emoji	`"hello  world  "`	—
合并空白	—	`"hello world"`

匹配算法使用**双向子串包含**而不是严格相等：如果回复文本包含了已发送的工具消息文本（或反过来），就视为重复。这捕获了模型在回复中"引用"工具消息内容的常见模式。

但注意 **10 字符最小长度门槛**：太短的文本（如 "ok"、"done"）不会参与去重，因为它们过于通用，容易产生误抑制。

在块输出中的检查点

去重检查发生在两个关键位置：

1. **`emitBlockChunk`** ——块流输出时（10.2 节中介绍的 BlockChunker 输出回调）：

```
// src/agents/pi-embedded-subscribe.ts（简化）
const emitBlockChunk = (text: string) => {
  if (state.suppressBlockChunks) return;
  const chunk = stripBlockTags(text, state.blockState).trimEnd();
  if (!chunk) return;

  // 去重检查
  const normalizedChunk = normalizeTextForComparison(chunk);
  if (isMessagingToolDuplicateNormalized(normalizedChunk, messagingToolSentTextsNormali
    log.debug(`Skipping block reply - already sent via messaging tool`);
    return;     // 静默丢弃
  }

  // ...正常输出...
  void params.onBlockReply({ text: cleanedText, ... });
};
```

2. **`handleMessageEnd`** ——消息结束时的最终回复发送：

```
// src/agents/pi-embedded-subscribe.handlers.messages.ts（简化）
// 在 message_end 处理中
const normalizedText = normalizeTextForComparison(text);
if (isMessagingToolDuplicateNormalized(normalizedText, ctx.state.messagingToolSentTexts
  ctx.log.debug(`Skipping message_end block reply - already sent via messaging tool`);
  // 跳过发送
} else {
  void onBlockReply({ text: cleanedText, ... });
}
```

这两个检查点确保了无论是流式分块还是完整消息结束，重复内容都不会泄漏给用户。

18.5.3 工具摘要内联

问题背景

当 AI Agent 执行工具（如读取文件、运行命令、搜索网页）时，用户在 Telegram、Discord 等通道中等待的过程可能很长。为了提供更好的体验，OpenClaw 会在工具执行时向用户发送**工具摘要（Tool Summary）**——一行简短的状态信息，告诉用户"Agent 正在做什么"。

摘要的触发与格式

工具摘要在 `handleToolExecutionStart` 中触发：

```
// src/agents/pi-embedded-subscribe.handlers.tools.ts（简化）
if (ctx.params.onToolResult && shouldEmitToolEvents && !ctx.state.toolSummaryById.has(t
  ctx.state.toolSummaryById.add(toolCallId);
  ctx.emitToolSummary(toolName, meta);
}
```

`emitToolSummary` 通过 `formatToolAggregate` 将工具名称和元信息格式化为一行可读文本：

```
// src/agents/pi-embedded-subscribe.ts（简化）
const emitToolSummary = (toolName?: string, meta?: string) => {
  const agg = formatToolAggregate(toolName, meta ? [meta] : undefined, {
    markdown: useMarkdown,
  });
  void params.onToolResult({ text: cleanedText, ... });
};
```

`formatToolAggregate` 的格式化逻辑

`formatToolAggregate`（位于 `src/auto-reply/tool-meta.ts`）是工具摘要的格式化核心。它将工具名映射到 emoji + 标签，并将元信息（通常是文件路径）进行智能分组：

```typescript
// src/auto-reply/tool-meta.ts（简化）
export function formatToolAggregate(
  toolName?: string,
  metas?: string[],
  options?: ToolAggregateOptions,
): string {
  const filtered = (metas ?? []).filter(Boolean).map(shortenMeta);
  const display = resolveToolDisplay({ name: toolName });
  const prefix = `${display.emoji} ${display.label}`;    // 如 "📖 Read"

  if (!filtered.length) return prefix;

  // 对路径进行目录分组和花括号折叠
  const grouped: Record<string, string[]> = {};
  for (const m of filtered) {
    if (!isPathLike(m)) { rawSegments.push(m); continue; }
    const parts = m.split("/");
    const dir = parts.slice(0, -1).join("/");
    const base = parts.at(-1) ?? m;
    if (!grouped[dir]) grouped[dir] = [];
    grouped[dir].push(base);
  }

  // 同目录下多个文件用花括号折叠
  const segments = Object.entries(grouped).map(([dir, files]) => {
    const brace = files.length > 1 ? `{${files.join(", ")}}` : files[0];
    return dir === "." ? brace : `${dir}/${brace}`;
  });

  return `${prefix}: ${meta}`;
}
```

产出的摘要示例：

工具调用	格式化结果
read(path="src/index.ts")	📖 Read: src/index.ts
exec(cmd="npm test", pty=true)	⚡ Exec: pty · npm test
read 多个同目录文件	📖 Read: src/{index.ts, config.ts}
message(action="send")	💬 Message: send

花括号折叠（brace collapse）是 shell 中常见的路径缩写语法——当同一目录下有多个文件时，用 `{file1, file2}` 替代逐个列出，使摘要更紧凑。

路径缩短

元信息中的文件路径会经过 `shortenMeta` 和 `shortenHomePath` 处理，将用户主目录替换为 `~`：

```
// src/auto-reply/tool-meta.ts
export function shortenMeta(meta: string): string {
  const colonIdx = meta.indexOf(":");
  if (colonIdx === -1) return shortenHomeInString(meta);
  const base = meta.slice(0, colonIdx);
  const rest = meta.slice(colonIdx);
  return `${shortenHomeInString(base)}${rest}`;
}
```

这意味着 `/Users/john/projects/app/src/index.ts` 会显示为 `~/projects/app/src/index.ts` ——既保留了足够的路径信息，又不会暴露系统用户名或浪费消息空间。

工具显示配置

`resolveToolDisplay`（位于 `src/agents/tool-display.ts`）从一个 JSON 配置文件 `tool-display.json` 中查找每个工具的显示信息：

```
// src/agents/tool-display.ts（简化）
export function resolveToolDisplay(params: {
  name?: string; args?: unknown; meta?: string;
}): ToolDisplay {
  const key = normalizeToolName(params.name).toLowerCase();
  const spec = TOOL_MAP[key];             // 从 JSON 配置中查找
  const emoji = spec?.emoji ?? "🧩";      // 默认 emoji
  const title = spec?.title ?? defaultTitle(name);
  const label = spec?.label ?? title;

  // 某些工具有动作特化的标签
  const actionSpec = resolveActionSpec(spec, action);
  const verb = normalizeVerb(actionSpec?.label ?? action);

  return { name, emoji, title, label, verb, detail };
}
```

这个设计将工具的**功能标识**与**显示表现**完全解耦——添加新工具时只需要在 JSON 配置中新增一条记录，无需修改代码逻辑。

详细输出模式

除了工具摘要，OpenClaw 还支持**工具输出内联**——将工具执行的完整输出嵌入到对话中。这由 `verboseLevel` 配置控制：

```
// src/agents/pi-embedded-subscribe.ts（简化）
const shouldEmitToolResult = () =>
  params.verboseLevel === "on" || params.verboseLevel === "full";

const shouldEmitToolOutput = () =>
  params.verboseLevel === "full";
```

详细程度	工具摘要	工具输出
`off`	❌	❌
`on`	✅	❌
`full`	✅	✅

在 `full` 模式下，工具输出通过 `emitToolOutput` 发送：

```
// src/agents/pi-embedded-subscribe.ts（简化）
const emitToolOutput = (toolName?: string, meta?: string, output?: string) => {
  const agg = formatToolAggregate(toolName, meta ? [meta] : undefined, { markdown: useM
  const message = `${agg}\n${formatToolOutputBlock(output)}`;
  void params.onToolResult({ text: cleanedText, ... });
};

const formatToolOutputBlock = (text: string) => {
  const trimmed = text.trim();
  if (!trimmed) return "(no output)";
  if (!useMarkdown) return trimmed;
  return `\`\`\`txt\n${trimmed}\n\`\`\``;      // 包裹在 Markdown 代码块中
};
```

这意味着在 `full` 模式下，用户会看到类似这样的消息：

📖 Read: src/config.ts
```txt
export const DEFAULT_PORT = 3000;
export const MAX_RETRIES = 3;
// ...
```

这对于调试和教学场景非常有用——用户可以实时看到 AI 正在读取什么文件、执行什么命令、得到什么结果。

本节小结 {#本节小结 }

1. **`NO_REPLY` 静默令牌**是一个简洁的"不回复"协议：模型输出 `NO_REPLY`，系统检测到后清空文本，上层
2. **消息工具去重**防止用户收到重复消息。采用"先暂存、后提交"的两阶段设计——只有工具执行成功后才记录已发
3. **工具摘要内联**通过 emoji + 标签 + 路径折叠，在用户等待工具执行时提供简洁的状态信息。路径自动缩短
4. **详细程度分级**（`off`/`on`/`full`）让用户可以按需选择信息密度——从完全静默到展示完整工具输出。
5. 这三个机制共同构成了回复的**最后一道过滤器**：过滤不该说的（静默）、删除重复说的（去重）、补充该看到的